

Executable Physical Objects: A Multilingual Architecture for Parametric Product Prototyping, Simulation, and Manufacturing

Abstract

Computer-aided design has traditionally treated the digital model as a geometric representation of a physical artefact. Contemporary engineering software has progressively extended this model through simulation, digital twins, generative design, manufacturing analysis, and real-time visualisation. Nevertheless, these capabilities frequently remain divided between specialised systems, file formats, and computational environments. This paper proposes an alternative architecture: the **Executable Physical Object (EPO)**, in which a product is represented not merely as geometry, but as a computational specification containing its dimensions, constraints, materials, manufacturing processes, tolerances, physical properties, and behavioural relationships.

The proposed system separates four computational concerns across complementary languages. Python provides the principal engineering and parametric modelling layer; Rust provides a high-performance computational substrate for geometry, spatial operations, caching, persistence, and runtime services; Kotlin provides the interactive engineering environment, real-time three-dimensional visualisation, animation, inspection, and human-computer interaction; and R provides statistical analysis, uncertainty modelling, process optimisation, and manufacturing-quality analysis. These layers communicate through explicitly defined engineering data structures and application interfaces rather than treating the graphical interface as the primary representation of the product.

The resulting architecture permits a product to be manipulated as a living engineering model. A change to a parameter such as wall thickness, diameter, material, or manufacturing process can propagate through geometry generation, physical calculations, tolerance analysis, manufacturing estimation, statistical evaluation, and visualisation. The same model can subsequently generate engineering visualisations, manufacturing specifications, inspection data, and machine-readable production formats.

The paper argues that this architecture represents a transition from **Computer-Aided Design (CAD)** toward **Executable Product Engineering (EPE)**. Rather than designing a representation of an object and subsequently analysing it, the proposed system treats the object itself as an executable engineering entity whose geometry, physical properties, manufacturing constraints, and economic characteristics are computationally coupled.

1. Introduction

The digital representation of a physical product has become central to modern engineering. Computer-aided design systems provide sophisticated mechanisms for constructing three-dimensional geometry, while finite-element analysis, computational fluid dynamics, manufacturing simulation, statistical process control, and digital-twin technologies increasingly allow engineers to reason about products before they exist physically.

Yet the contemporary engineering workflow remains substantially fragmented.

A designer may construct a component in one application, export the geometry into another application for simulation, transfer dimensions into a manufacturing system, analyse production tolerances using statistical software, and finally construct a separate visualisation or interactive demonstration. Each stage operates upon a representation of the product, but the representations are not necessarily identical.

This produces a fundamental architectural problem:

The digital model of a physical object is frequently a representation of the object rather than the object's complete engineering specification.

This paper proposes a different approach.

The central proposition is that a physical product should be represented as an **executable object model**. Geometry is one component of that model, but it is not its entirety. A product should simultaneously describe:

- geometric dimensions;
- parametric relationships;
- material properties;
- manufacturing processes;
- dimensional tolerances;
- physical constraints;
- assemblies and component relationships;
- cost and production characteristics;
- inspection requirements;
- simulation parameters; and
- exportable manufacturing representations.

Under this approach, changing a parameter is not simply an operation on a mesh. It is an operation on the engineering definition of the product.

For example, consider a ceramic mug specified as:

```
height = 105 mm  
diameter = 82 mm  
wall_thickness = 3.2 mm  
material = stoneware  
manufacturing_process = slip_casting
```

A conventional visual modelling system might use these values to construct geometry.

An executable physical-object system would additionally determine the consequences of those values. Altering wall thickness could change the model's mass, thermal behaviour, structural characteristics, material consumption, estimated production cost, manufacturing yield, and tolerance requirements. The graphical representation would therefore become one output of an underlying engineering computation rather than the primary source of truth.

2. Research Proposition

The proposed research investigates the following question:

Can a multilingual software architecture represent physical products as executable engineering objects, allowing geometry, simulation, manufacturing, statistical analysis, and interactive visualisation to remain synchronised throughout the product-development lifecycle?

The hypothesis is that such an architecture can reduce representational fragmentation and provide a more coherent computational basis for rapid physical prototyping.

The system is therefore not proposed as a replacement for established CAD, CAM, CAE, or manufacturing systems in its initial form. Rather, it is proposed as an architectural framework through which these traditionally separate capabilities can be progressively unified around a common executable product representation.

3. The Executable Physical Object

An EPO can be conceptualised as:

$$EPO = G + P + M + T + S + F + C$$

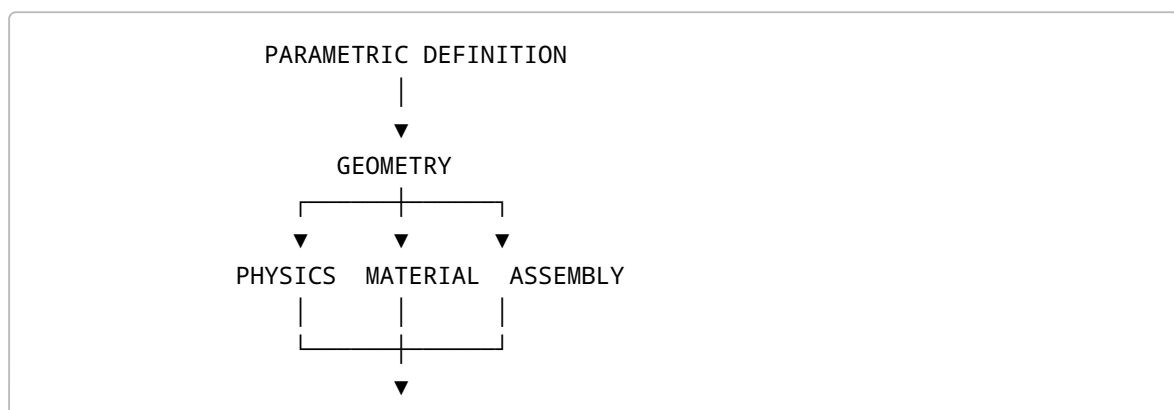
where:

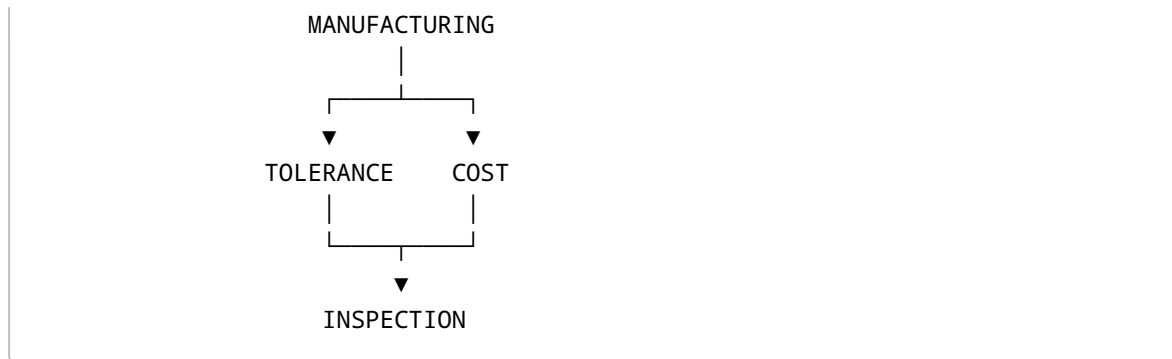
- G = geometry;
- P = physical properties;
- M = manufacturing specification;
- T = tolerances and constraints;
- S = simulation state;
- F = functional relationships; and
- C = cost and production characteristics.

The essential distinction is that these elements are computationally related.

A change in G may therefore propagate into P , M , T , S , and C .

This creates a dependency graph rather than a static model:





The model consequently becomes an executable dependency network.

4. A Multilingual Engineering Architecture

The proposed implementation deliberately avoids attempting to force every computational task into a single programming language.

Instead, each language is assigned a specific architectural responsibility.

4.1 Python: Engineering Semantics

Python forms the principal engineering layer.

It provides:

- parametric modelling;
- constraint evaluation;
- geometry construction;
- engineering calculations;
- manufacturing rules;
- simulation orchestration;
- validation;
- model generation; and
- high-level scripting.

Python is particularly appropriate for the experimental layer because engineering algorithms can be rapidly developed, tested, and modified.

4.2 Rust: Computational Substrate

Rust provides the performance-critical foundation.

It can implement:

- geometry kernels;
- spatial indexing;
- collision detection;
- mesh processing;
- Boolean operations;
- tessellation;

- parallel computation;
- caching;
- project persistence; and
- interprocess communication.

Python can therefore operate as the expressive engineering layer while Rust handles computationally intensive operations.

4.3 Kotlin: Interactive Engineering Environment

Kotlin provides the human-facing engineering environment.

Rather than treating the graphical interface as a conventional CAD front end, the Kotlin layer becomes an interactive engineering workstation capable of:

- real-time three-dimensional rendering;
- parametric manipulation;
- animated assemblies;
- exploded views;
- section views;
- dimensional inspection;
- manufacturing visualisation;
- simulation playback;
- telemetry;
- engineering dashboards; and
- interactive design exploration.

This creates an important separation between **engineering computation** and **engineering experience**.

4.4 R: Statistical and Manufacturing Intelligence

R provides the statistical layer.

Potential applications include:

- tolerance analysis;
- Monte Carlo simulation;
- manufacturing variation;
- process capability;
- production yield;
- experimental design;
- regression;
- reliability modelling;
- cost optimisation; and
- quality-control analysis.

Consequently, the system can move beyond asking whether a product *can* be manufactured toward estimating how reliably and economically it can be manufactured.

5. From CAD to Executable Product Engineering

The proposed architecture suggests a broader conceptual transition.

Traditional CAD can be represented approximately as:

$$Design \rightarrow Geometry$$

The proposed architecture instead becomes:

$$Design \rightarrow Geometry \rightarrow Physics \rightarrow Manufacturing \rightarrow Statistics \rightarrow Cost \rightarrow Inspection \rightarrow Production$$

The digital object consequently becomes a common computational reference point throughout the product lifecycle.

This may be understood as **Executable Product Engineering (EPE)**.

The principal research proposition is therefore not that another three-dimensional modelling application should be constructed, but that the fundamental unit of product-development software could be reconsidered.

The unit of computation is no longer the drawing, mesh, or CAD file.

It is the **physical object as an executable engineering system**.

6. Research Programme

An initial implementation could evaluate the architecture using progressively complex objects.

Phase I — Parametric Objects

Simple manufactured products such as:

- mugs;
- bottles;
- housings;
- brackets; and
- mechanical components.

Phase II — Assemblies

Multi-component products including:

- pumps;
- motors;
- mechanisms;
- appliances; and
- small machines.

Phase III — Manufacturing Simulation

The system would model:

- machining;
- casting;
- additive manufacturing;
- injection moulding;
- sheet-metal fabrication; and
- assembly.

Phase IV — Statistical Manufacturing

R-based analysis would introduce:

- tolerance propagation;
- manufacturing variation;
- process capability;
- yield estimation; and
- production optimisation.

Phase V — Physical Validation

Digital models would be compared with manufactured prototypes using dimensional inspection and metrology data.

The central experimental question would then become:

How accurately can the executable product model predict the physical product?

7. Conclusion

The proposed architecture treats product engineering as a computationally unified discipline rather than a sequence of disconnected software operations.

Python provides engineering expressiveness. Rust provides computational performance. Kotlin provides an interactive spatial environment. R provides statistical and optimisation capabilities.

Together they form an architecture in which a physical product can be specified, visualised, simulated, analysed, optimised, inspected, and prepared for manufacture from a common computational definition.

The long-term objective is therefore not merely a better CAD system.

It is a system in which a product can be **programmed before it is manufactured**.

Such a system could provide a foundation for a new generation of engineering software in which the boundary between CAD, simulation, manufacturing engineering, statistical quality control, and interactive visualisation becomes increasingly fluid.

The fundamental proposition can be stated simply:

The next generation of CAD should not merely draw physical objects. It should execute them.